PRODUCT REQUIREMENTS DOCUMENT (PRD)

CodeSphere – CSE Coding Learning & Assessment Platform

Project: CodeSphere – CSE Coding Learning & Assessment Platform

Initial Learning Language: C Programming

Target Users: CSE Coding Club students and administrators

Expected Concurrent Usage: Approximately 60 students

1. Product Overview

CodeSphere is a web-based learning and coding assessment platform designed for the CSE Coding Club. The first release focuses on C Programming and combines structured learning resources with an online coding environment similar in concept to competitive programming platforms.

The platform enables students to learn through curated resources, practice programming problems, participate in timed coding rounds, submit code for automated evaluation, and track their progress. Administrators can manage students, learning content, coding problems, assessments, and results.

2. Problem Statement

College coding clubs often manage learning materials, practice questions, coding tests, submissions, and results using multiple disconnected tools. This makes it difficult to provide a unified learning experience and to monitor student progress.

CodeSphere addresses this problem by providing a centralized platform for learning C programming and conducting coding assessments for approximately 60 students concurrently.

3. Goals

• Provide a centralized C programming learning dashboard.

• Provide curated YouTube reference videos and topic-based learning modules.

• Provide coding practice problems with an online code editor.

• Conduct timed coding rounds similar to simplified LeetCode/HackerRank workflows.

• Evaluate C programs automatically using Judge0.

• Automatically save submissions and provide results.

• Support approximately 60 concurrent students.

• Provide administrator controls for content, questions, rounds, and results.

• Provide student progress tracking and leaderboards.

4. Non-Goals for Version 1

• Supporting many programming languages in the initial release.

• Building a custom C compiler or directly executing untrusted code on the main backend server.

• Advanced AI proctoring or webcam monitoring.

• Full-scale nationwide competitive programming infrastructure.

5. User Roles

Role	Responsibilities
Student	Login, access learning modules, watch reference videos, practice coding, attend coding rounds, submit solutions, view results and progress.
Admin / Coding Club Coordinator	Manage students, learning resources, problems, test cases, coding rounds, schedules, results and leaderboard.

6. Core Features

• Authentication using JWT.

• Student and administrator dashboards.

• C programming learning path organized by modules and topics.

• YouTube resource links embedded or linked from learning topics.

• Problem bank with title, statement, examples, constraints, difficulty and test cases.

• Monaco Editor-based coding interface.

• Run Code and Submit Code actions.

• Judge0 integration for safe compilation and execution.

• Timed coding rounds with multiple questions.

• Autosave of student code during an active assessment.

• Assessment submission when time expires.

• Page visibility/focus event logging and configurable auto-submit policy when the assessment page is left.

• Automatic result evaluation and score calculation.

• Student progress dashboard and leaderboard.

• Admin panel for managing assessments and content.

7. Functional Requirements

ID	Requirement
FR-01	The system shall allow authorized students and administrators to log in.
FR-02	The student dashboard shall display learning progress and available coding activities.
FR-03	The system shall organize C programming content into ordered modules and topics.
FR-04	Each learning topic may include curated YouTube reference resources.
FR-05	Students shall be able to open coding problems and write C programs in an online editor.

FR-06	Students shall be able to run code against permitted sample/custom inputs.
FR-07	Students shall be able to submit code for automated evaluation.
FR-08	The backend shall send code execution jobs to Judge0 rather than executing untrusted code locally.
FR-09	The system shall evaluate outputs against configured test cases and store submission results.
FR-10	Admins shall be able to create timed coding rounds and assign problems.
FR-11	The coding round shall display a countdown timer.
FR-12	The system shall autosave active assessment code at defined intervals.
FR-13	When assessment time expires, the latest saved code shall be submitted automatically.
FR-14	The system shall detect browser visibility changes and window focus changes while an assessment is active.
FR-15	Depending on the configured assessment policy, a visibility violation may trigger automatic submission and assessment locking.
FR-16	The system shall store scores and submission history.
FR-17	Students shall be able to view their own results and progress.
FR-18	Admins shall be able to view student performance and round results.

8. Non-Functional Requirements

• Concurrent usage: target approximately 60 active students.

• Performance: normal dashboard and API actions should respond quickly under expected load.

• Reliability: autosave should reduce loss of student work during network interruptions.

• Security: passwords must be hashed; protected APIs must require authentication and authorization.

• Scalability: code execution must be separated from the main application backend.

• Availability: deployment should avoid sleeping/cold-start issues during scheduled coding rounds.

• Usability: responsive interface suitable for desktop browsers used in college labs.

• Maintainability: modular frontend and backend architecture with documented APIs.

• Data integrity: submissions, timestamps and scores must be stored consistently.

9. Success Criteria

• At least 60 students can access the platform during a coding club session.

• Students can complete a C programming learning path and open practice problems.

• Students can run and submit C code successfully.

• Timed coding rounds correctly track remaining time and submit on expiry.

• Results are linked to the correct authenticated student.

• Administrators can create and review coding rounds without manual result checking.

10. MVP Scope

The first deployable version will focus on the essential workflow: authentication → C learning dashboard → coding practice → timed coding round → Judge0 evaluation → result storage → leaderboard.

- Student/Admin login
- C programming modules + YouTube references
- Problem bank
- Monaco Editor
- Run/Submit
- Judge0 execution
- Timed coding rounds
- Autosave
- Visibility/focus event handling
- Results and leaderboard